

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ

з лабораторної роботи № 4
з дисципліни «Основи програмування»
«Основні типи змінних. Операції та їх пріоритет»

Виконав:
студент групи ФЕП-13
Марцінко Н. Р.

Перевірив:
асистент Васюта В. М.

Львів – 2026

Мета роботи

Вивчити поняття: основні типи змінних, вказівники, операції та їх пріоритет.

1. Завдання 1. Арифметичні, логічні та побітові оператори

Програма демонструє арифметику для $a = 10$, $b = 3$, логічні операції для $x = 1$, $y = 0$ та побітові операції у двійковому вигляді (%08b).

1.1. Текст програми (task1.c)

```
#include <stdio.h>

int main() {
    int a = 10, b = 3;

    printf("a = %d, b = %d\n", a, b);
    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    printf("a %% b = %d\n", a % b);
    printf("++b = %d\n", ++b);

    int x = 1, y = 0;
    printf("x = %d, y = %d\n", x, y);
    printf("x && y = %d\n", x && y); // I
    printf("x || y = %d\n", x || y); // АБО
    printf("!x = %d\n", !x); // НЕ

    unsigned char p = 0b00001100; // 12
    unsigned char q = 0b0000101; // 5

    printf("p = %08b\n", p);
    printf("q = %08b\n", q);

    printf("p & q = %08b\n", p & q); // i
    printf("p | q = %08b\n", p | q); // або
    printf("p ^ q = %08b\n", p ^ q); // виключне або
    printf("~p = %08b\n", (unsigned char)~p); // заперечення
    printf("p << 2 = %08b\n", p << 2); // зсув вліво
    printf("p >> 1 = %08b\n", p >> 1); // зсув вправо

    return 0;
}
```

Рис. 1. Код програми task1.c

2.2. Результат виконання

```

a = 10, b = 3
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1
++b = 4
x = 1, y = 0
x && y = 0
x || y = 1
!x = 0
p = 00001100
q = 00000101
p & q = 00000100
p | q = 00001101
p ^ q = 00001001
~p = 11110011
p << 2 = 00110000
p >> 1 = 00000110

```

Рис. 2. Вивід task1

Арифметика: $10+3=13$, $10-3=7$, $10*3=30$, $10/3=3$, $10\%3=1$, $++b$ дає 4. Логіка: $1\&0=0$, $1||0=1$, $!1=0$. Побітово для $p=00001100$, $q=00000101$: $\& \rightarrow 00000100$, $| \rightarrow 00001101$, $\wedge \rightarrow 00001001$, $\sim p \rightarrow 11110011$, зсуви $<<2$ і $>>1$ працюють як очікувалось.

3. Завдання 2. Вказівники

Зчитується число в змінну `a`, створюється вказівник `ptr = &a`. Виводяться значення `a`, адреса `a`, адреса, що зберігає `ptr`, і значення через `*ptr`.

3.1. Текст програми (task2.c)

```

#include <stdio.h>

int main() {
    int a;

    printf("Введіть значення a: ");
    scanf("%d", &a);
    int *ptr = &a;
    printf("Значення a: %d\n", a);
    printf("Адреса a: %p\n", &a);
    printf("Адреса пам'яті за вказівником: %p\n", ptr);
    printf("Значення за вказівником: %d\n", *ptr);

    return 0;
}

```

Рис. 3. Код програми task2.c

3.2. Результат виконання

```

Введіть значення a: 576575
Значення a: 576575
Адреса a: 0x7ffda26a77cc
Адреса пам'яті за вказівником: 0x7ffda26a77cc
Значення за вказівником: 576575

```

Рис. 4. Вивід task2 (введено 576575)

Адреса `a` і значення `ptr` збігаються (`0x7ffda26a77cc`), `*ptr` повертає те саме `576575` — вказівник коректно посилається на змінну.

4. Завдання 3. Квадратне рівняння

Коефіцієнти `a`, `b`, `c` вводяться з клавіатури. Обчислюється дискримінант $d = b^2 - 4ac$. Залежно від знака `d` виводяться корені (через `sqrt` з `math.h`) або повідомлення про відсутність дійсних коренів.

4.1. Текст програми (task3.c)

```

#include <stdio.h>
#include <math.h>

int main() {
    int a, b, c;
    double d, x1, x2;
    printf("Введіть a, b, c: ");
    scanf("%d %d %d", &a, &b, &c);
    d = b * b - 4 * a * c;
    if (d == 0) {
        x1 = -b / (2 * a);
        printf("%lf\n", x1);
    }
    else if (d > 0) {
        x1 = (-b - sqrt(d)) / (2 * a);
        x2 = (-b + sqrt(d)) / (2 * a);
        printf("%lf %lf\n", x1, x2);
    }
    else {
        printf("Рівняння не має дійсних коренів\n");
    }
    return 0;
}

```

Рис. 5. Код програми task3.c

4.2. Результат виконання

```

Введіть a, b, c: 34 43 6
-1.105005 -0.159701

```

Рис. 6. *Visual task3* ($a=34$, $b=43$, $c=6$)

Для $a=34$, $b=43$, $c=6$ дискримінант додатний; корені ≈ -1.105 і -0.160 .

Висновки

У результаті виконання лабораторної роботи №4 реалізовано та протестовано програми мовою C. Практичне використання стандартних функцій введення/виведення (`printf`, `scanf`) укотре продемонструвало недоліки архітектури C.

Головна проблема полягає у відсутності строгого контролю типів при форматованому виводі. Наприклад, при розв'язанні квадратного рівняння через дискримінант випадкова плутанина у специфікаторах замість логічної помилки компіляції призводить до того, що у відповідь я отримую сміття з чисел.

Окремого зауваження заслуговує еволюція самої мови: розробникам знадобилося майже 50 років, щоб нарешті додати в `printf/scanf` специфікатор `%b` для двійкового формату, але при цьому в C досі немає окремого нативного типу для байта (доводиться використовувати застарілий `unsigned char`).